Jenkins-Task -4

1) Setup Jenkins CICD pipeline using freestyle job using Docker containers using below code.
Source Code: https://github.com/betawins/hiring-app.git
Stages:

- Git Clone
- SonarQube Integration
- Maven Compilation
- Nexus Artifactory
- Slack Notification
- Deploy On tomcat

Runtime Environment: Java 21 (Amazon Corretto)
sudo dnf install java-21-amazon-corretto -y

Stage-1
Git Clone
New Item → Hiring_App→ Freestyle → Source Code Management → Git → installed git in Ec2 Jenkins master server also → Save → Build Now

Stage-2
SonarQube Integration:
→ sudo yum -y install docker
→ systemctl start docker
→ docker pull sonarqube
→ docker images
→ docker run -itd -p 9002:9000 sonarqube
→ docker ps

To Access : Ec2 pub_ip:9002

Jenkins Settings
Jekins → Dashboard → Manage Jenkins → Systems → SonarQube servers →
Name: jenkins_1
Server: URL: http://18.212.205.252:9002
Server authentication token: **********

save
Server Authentication Token:
Jenkins → Manage Jenkins → Credentials → System → Global

Stage-3
Maven
cd /opt/
wget https://dlcdn.apache.org/maven/maven-3/3.10.0/binaries/apache-maven-3.10.0-bin.tar.gz
sudo tar -xvf apache-maven-3.10.0-bin.tar.gz
Edit → vi .bash_profile

Jenkins-Task -4

MVN_HOME= /opt/apache-maven-3.10.0/bin
PATH=$PATH:$HOME/bin:$MVN_HOME

Export PATH

Save; :wq!
check: echo $MVN_HOME
→ mvn –version

Jenkins Settings
Jenkins → Manage Jenkins → Tools → Maven Installation

Name: MVN_HOME

MAVEN_HOME: /opt/apache-maven-3.10.0

In JOB Configuration
Dashboard → Hiring_App→ Freestyle → Configure → Post-build Actions → ADD build Step
→ Invoke top-level Maven targets
→ Maven Version : MVN_HOME
→ Goals : clean install
→ save → Build Now

Then check in this after Build
#There will be one Target Directory Will created after running MAVEN
/var/lib/jenkins/workspace/Hiring_App/target
→ This file will Appear = hiring.war
→ cd /var/lib/Jenkins/.m2/repository
Here We can Find "Repository" = In This we can find Maven Dependencies of that Repo

Stage-4
Nexus Artifactory
cd /opt/
→ wget https://download.sonatype.com/nexus/3/nexus-3.96.4-01-linux-x86_64.tar.gz
→ sudo tar -zxvf nexus-3.96.4-01-linux-x86_64.tar.gz
→ sudo mv nexus-3.96.4-01-linux-x86_64 nexus

Then: sudo vi /opt/nexus/bin/nexus.rc
Set: run_as_user="ec2-user"

Change ownership to ec2-user
sudo chown -R ec2-user:ec2-user /opt/nexus
Also:
sudo chown -R ec2-user:ec2-user /opt/sonatype-work
Then verify:

Jenkins-Task -4

ls -ld /opt/nexus
ls -ld /opt/sonatype-work
Start: ./nexus start
sudo ss -lntp | grep 8081

==========

Troubleshoot if Any thing Happen

Check the main Nexus log: tail -100 /opt/sonatype-work/nexus3/log/nexus.log
If that file doesn't exist, run: ls -lah /opt/sonatype-work/nexus3/log/
Check all Nexus log files: ls -lah /opt/sonatype-work/nexus3/log/
Check whether a Nexus process exists: ps -ef | grep nexus
and: ps -ef | grep java
Check the JVM startup error: cat /opt/sonatype-work/nexus3/log/jvm.log
Check whether Nexus created any other logs:
→ find /opt/sonatype-work/nexus3 -maxdepth 3 -type f -ls
Check the Nexus Java/runtime configuration: cd /opt/nexus/bin → ./nexus run

If it is related to storage Problem
Nexus itself is fine; the EC2 instance does not have enough RAM for the JVM configuration.
Check your instance memory: free -h and df -h /
Check the Nexus JVM configuration: cat /opt/nexus/bin/nexus.vmoptions
The important lines will probably look similar to:
-Xms2703m
-Xmx2703m
Reduce Nexus JVM memory:
sudo sed -i 's/-Xms2703m/-Xms512m/' nexus.vmoptions
sudo sed -i 's/-Xmx2703m/-Xmx512m/' nexus.vmoptions

Then verify:
grep -E '^-Xms|^-Xmx' /opt/nexus/bin/nexus.vmoptions
You should see:
-Xms512m
-Xmx512m
Start Nexus in foreground
→ cd /opt/nexus/bin
→ ./nexus run
Wait around **30–60 seconds**.

If it starts successfully, you should eventually see startup messages rather than the previous
If it starts successfully → Do not press Ctrl+C yet
Open another SSH terminal to the Nexus EC2 and run: sudo ss -lntp | grep 8081
Then: curl -I http://localhost:8081
In Browser: http://Ec2 pub_ip:8081
Username: admin

Jenkins-Task -4

For Password Run this on Ec2: cat /opt/sonatype-work/nexus3/admin.password
Password: 858720b8-7f7d-452b-a304-8b0fb0f6e555

==========

Jenkins Settings
Jenkins → Manage Jenkins → Plugins → Available plugins → Nexus Artifact Uploader

Configure in Job
Jenikins → Hiring_Job → Configure → Build Steps → Add Build Step
→ select → Nexus Artifact Uploader

In Nexus Artifact Uploader
Nexus Version : Nexus3
Protocol: HTTP
Nexus URL: 54.159.123.212:8081
Credentials add: Add → Global → Username With Password
Username: same as Nexus Server: Admin
Password: same as Nexus Server: **********
id: Any Name
Description: Any Discription
Group id: Find in Pom.Xml
Version: Find in Pom.Xml
Repository: First Create a repo in Nexus Server { Hiring_App }
Nexus Repository 3 (Go to) : Settings → Repositories → Create repository
Then choose the Maven format you need: maven2 (hosted) → After Creating
→ copy the same repo name from Nexus → Paste it in Jenkins → Hiring_App

Then Click → Artifacts → Add
ArtifactId: hiring
Type: war
Classifier:
File: /var/lib/jenkins/workspace/Hiring_App/target/
Jenkins automatically uses the workspace belonging to **that particular job**. = target/*.war
Since you have only one WAR File: target/hiring.war
In pom.xml = <finalName>hiring</finalName>

Stage-5
Slack Notification
Section 5: Slack Integration Guide
Step 5.1: Create a Slack Channel

1. Go to slack.com and make a free workspace named **DevOps-Test**.
2. Open the workspace. In the left sidebar, click **Add channels** ➡️ **Create a new channel**.
3. Name the channel **jenkins-builds**.

Jenkins-Task -4

4. Make sure it is set to **Public**, then click **Create**.

Step 5.2: Create the Slack App Bot

1. Open a new browser tab and go to slack.com.
2. Click **Create New App**, and select **From scratch**.
3. Name the app **Jenkins Bot** and select your **DevOps-Test** workspace. Click **Create App**.
4. In the left menu, click **OAuth & Permissions**.
5. Scroll down to the **Bot Token Scopes** section, click **Add an OAuth Scope**, and add these two permissions:
    - **chat:write**
    - **chat:write.public**
6. Scroll back to the top of the page and click the green **Install to Workspace** button.
7. Click **Allow** on the permission screen.
8. Copy the **Bot User OAuth Token** that appears (it starts with **xoxb-**).

Step 5.3: Add the Token to Jenkins

1. Open your **Jenkins Dashboard** tab.
2. Go to **Manage Jenkins** ➡️ **Credentials** ➡️ **(global)** ➡️ **Add Credentials** (top right).
3. Fill out the fields exactly like this:
    - **Kind:** Secret text
    - **Secret:** Paste your **xoxb-** token.
    - **ID:** slack-token
    - **Description:** Slack Bot Token
4. Click **Create**.

Step 5.4: Configure Jenkins System Settings

1. Go to **Manage Jenkins** ➡️ **System**.
2. Scroll to the bottom to find the **Slack** section.
3. Fill out the configuration exactly like this:
    - **Workspace:** devops-test
    - **Credential:** Select **slack-token** from the dropdown menu.
    - **Default channel:** #jenkins-builds
    - **Custom slack app bot user: CHECK THIS BOX**.
4. Click **Test Connection**. You will instantly see a **"Success"** message appear in your Slack channel.
5. Scroll to the bottom of the Jenkins page and click **Save**.

Step 5.5: Enable Slack Alerts in your Job

1. Open your project job (**stage_1**) and click **Configure**.
2. Scroll to the absolute bottom to the **Post-build Actions** section.
3. Click **Add post-build action** and choose **Slack Notifications**.
4. Check the boxes for **Notify Success** and **Notify Failure**.
5. Click **Save**.

Step 5.6: Run a Test Build

Jenkins-Task -4

Stage-6

Deploy On tomcat

Plugin Installation:

Jekins → Manage Jenkins → Plugins → Available plugins → Deploy to container → install

Job Configuration

Dashboard → Hiring_App → Configue → Post-build Actions →

Add post-build action → Deploy war/ear to a container

WAR/EAR files: **/*.war

Context path:

Containers: Tomcat 9.x Remote

Credentials : ADD

Username with Password

Username: deployer

Password: deployer

ID: Any name

Description: Any thing

In Tomcat Ec2-Server

Before this Credentials : SSh to tomat Server

→ find / -name context.xml

/opt/apache-tomcat-9.0.122/conf/context.xml

/opt/apache-tomcat-9.0.122/webapps/docs/META-INF/context.xml

/opt/apache-tomcat-9.0.122/webapps/examples/META-INF/context.xml

/opt/apache-tomcat-9.0.122/webapps/host-manager/META-INF/context.xml

/opt/apache-tomcat-9.0.122/webapps/manager/META-INF/context.xml

edit only manager files with the help of vi editor

vi /opt/apache-tomcat-9.0.122/webapps/host-manager/META-INF/context.xml

```
<!-- <Valve className="org.apache.catalina.valves.RemoteCIDRValve"
allow="127.0.0.0/8,::1/128" /> -->
```

Vi /opt/apache-tomcat-9.0.122/webapps/manager/META-INF/context.xml

```
<!-- <Valve className="org.apache.catalina.valves.RemoteCIDRValve"
allow="127.0.0.0/8,::1/128" /> -->
```

Then : cd /opt/ apache-tomcat-9.0.122

cd Conf/ → vi tomcat-users.xml → add these

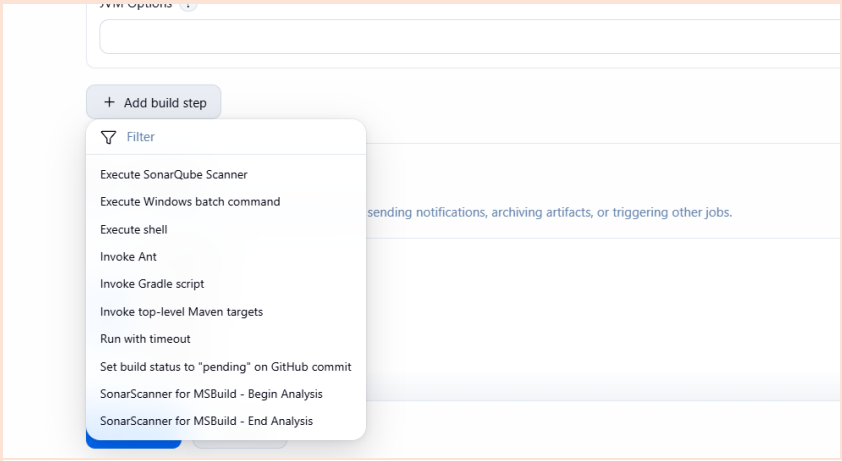
```
<role rolename="manager-gui"/>
        <role rolename="manager-script"/>
        <role rolename="manager-jmx"/>
        <role rolename="manager-status"/>
```

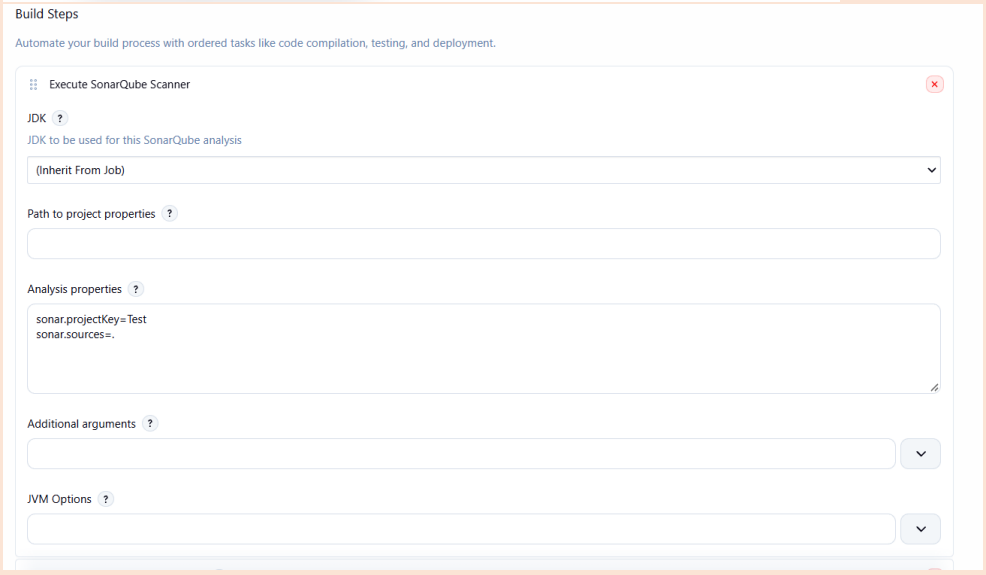
Jenkins-Task -4

<user username="admin" password="admin" roles="manager-gui, manager-script, manager-jmx, manager-status"/>
<user username="deployer" password="deployer" roles="manager-script"/>
<user username="tomcat" password="s3cret" roles="manager-gui"/>

Implementtaion:

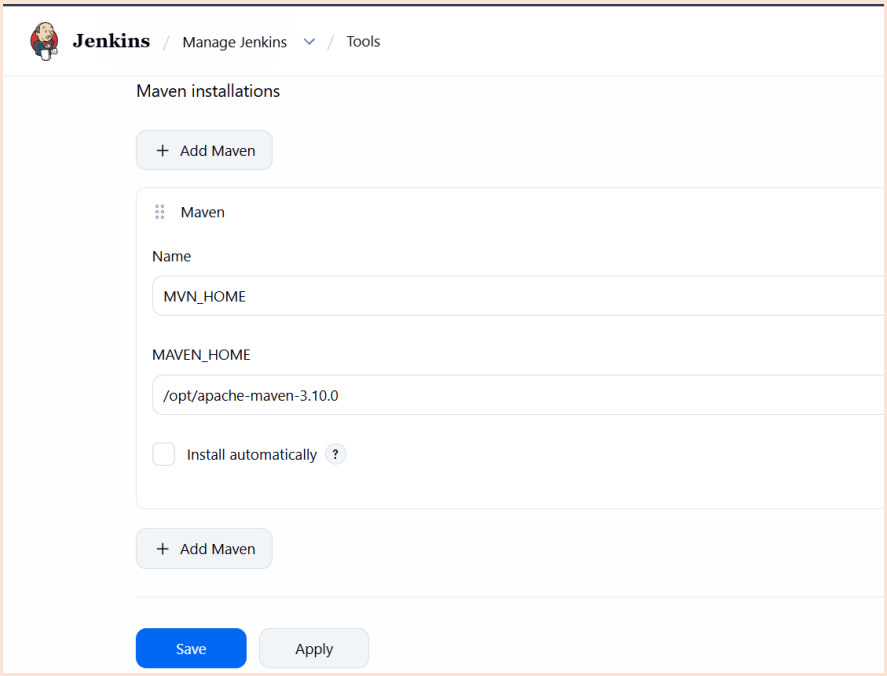
Stage-2

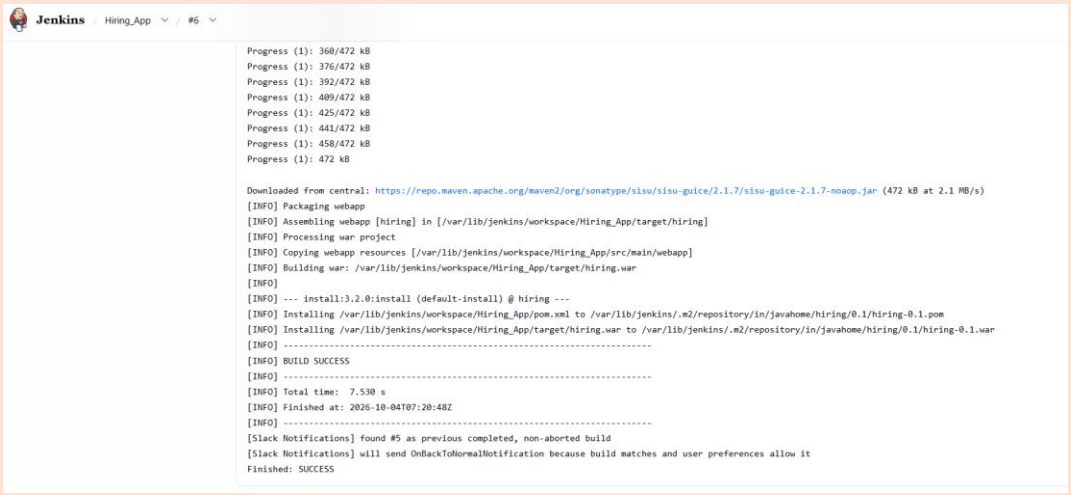




Jenkins-Task -4

Stage-3

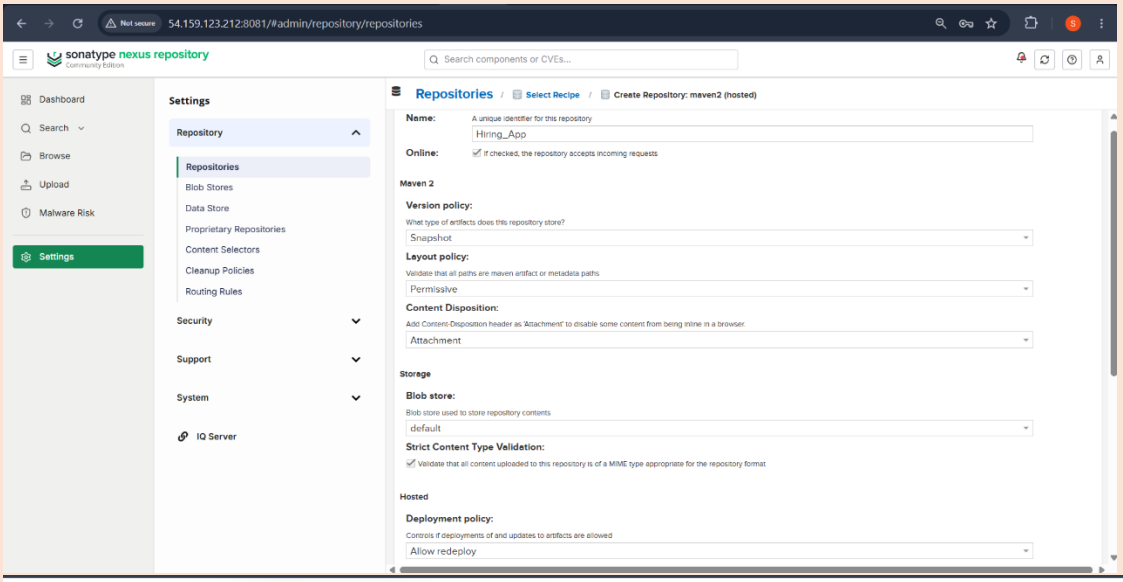




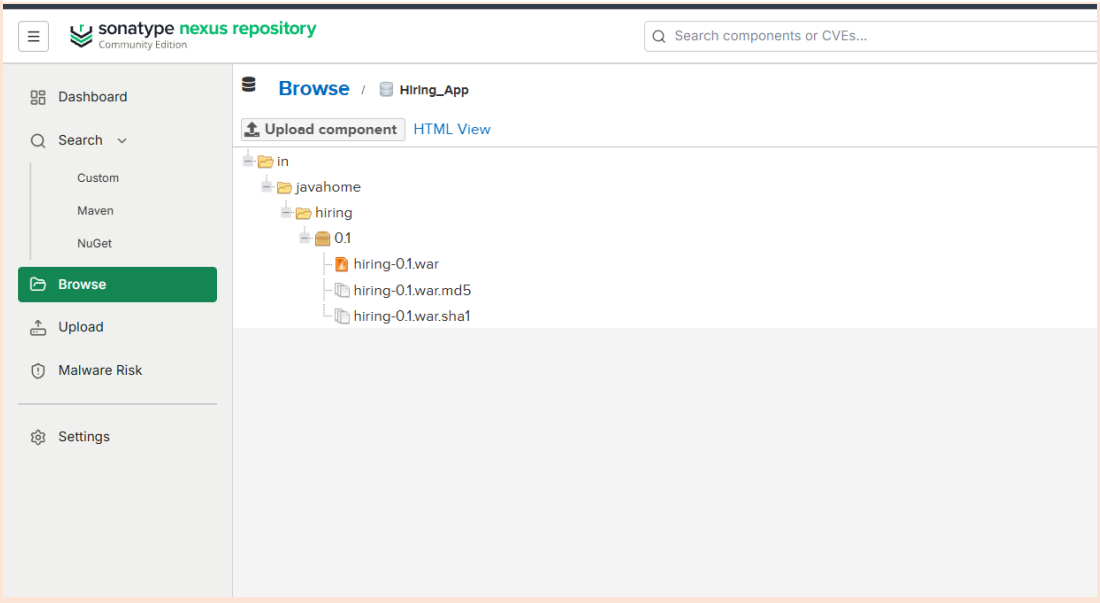

```
[ec2-user@ip-172-31-29-21 workspace]$ ll
total 16
drwxr-xr-x. 6 jenkins jenkins 16384 Oct  4 07:20 Hiring_App
[ec2-user@ip-172-31-29-21 workspace]$ cd Hiring_App/
[ec2-user@ip-172-31-29-21 Hiring_App]$ ls
 Dockerfile   Jenkinsfile   README.md  'Untitled Diagram.drawio'   jenkinsfile-cicd   pom.xml   src   target
[ec2-user@ip-172-31-29-21 Hiring_App]$ cd target/
[ec2-user@ip-172-31-29-21 target]$ ls
hiring  hiring.war  maven-archiver
[ec2-user@ip-172-31-29-21 target]$
```

Stage-4

Jenkins-Task -4

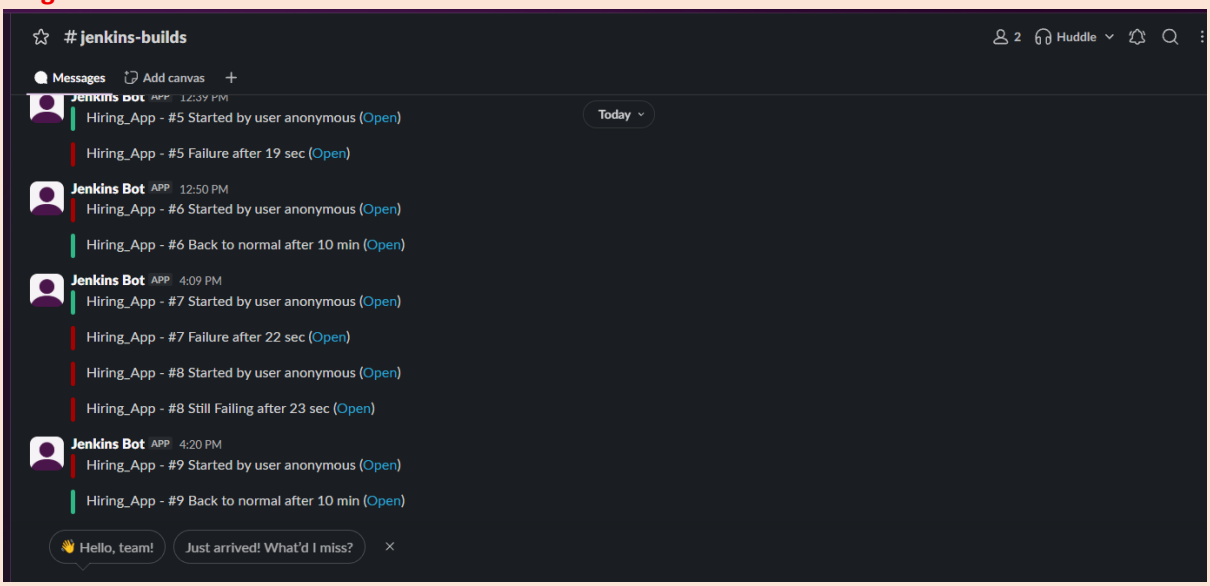




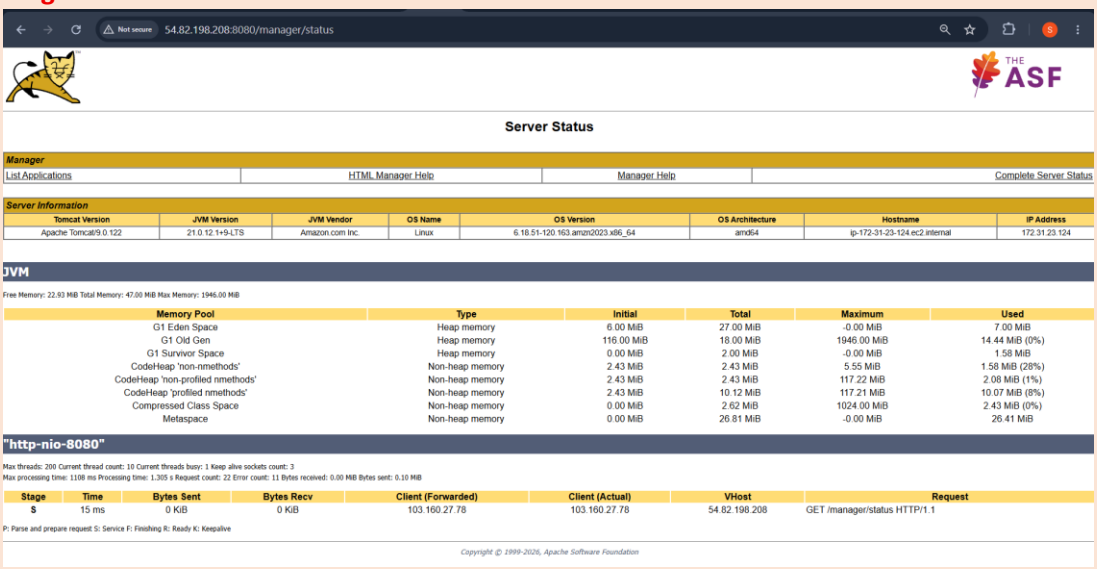


Jenkins-Task -4

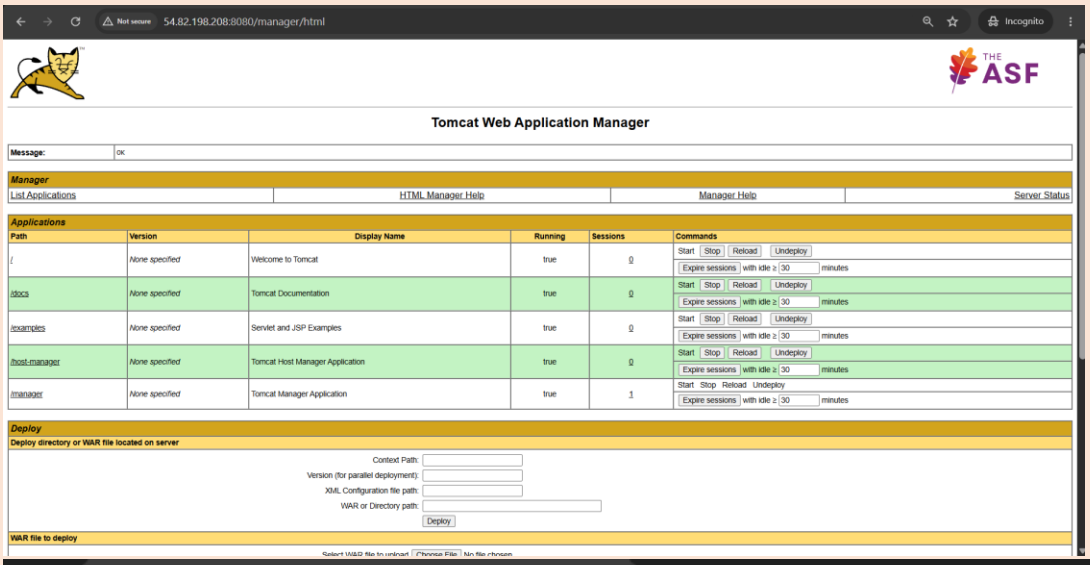
Stage-5

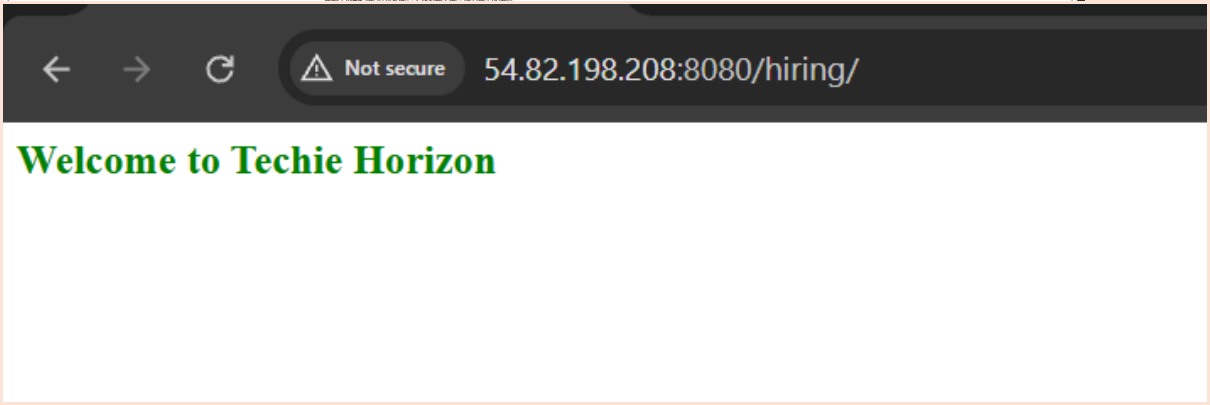


Stage-6



Jenkins-Task -4





2) Setup a Jenkins CICD pipeline using Declarative pipeline using feature-1.1 branch.

Source Code: https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

stages:

- Git Clone
- SonarQube Integration
- Maven Compilation
- Nexus Artifactory
- Slack Notification
- Deploy On tomcat

Objective:

To implement a complete Declarative Jenkins Pipeline that automates the build, quality analysis, artifact storage, notification, and deployment lifecycle of the SimpleCustomerApp Java web application using the following six stages:

Jenkins-Task -4

Git Clone → SonarQube Integration → Maven Compilation → Nexus Artifactory → Slack Notification → Deploy On Tomcat

Purpose

The purpose of this task is to demonstrate a fully automated CI/CD workflow in Jenkins using Declarative Pipeline syntax. Each stage of the pipeline is validated independently and confirmed in both the Jenkins console and the corresponding external system (SonarQube, Nexus, Slack, and Tomcat). The task also demonstrates best practices such as credential handling, environment configuration, and troubleshooting of real production-style failures.

Implementation — Sequential Procedure

Step 1 — Repository Preparation

The upstream repository betawins/sabear_simplecutomerapp was forked to Sunith3210/sabear_simplecutomerapp. A pom.xml fix was applied and committed on the feature-1.1 branch to make Maven package the WebContent/ directory (see Section 9 for details).

Step 2 — Jenkins Job Creation

1. Jenkins → New Item → Name: SimpleCustomerApp → Type: **Pipeline** → OK.
2. Configure → Pipeline → Definition: **Pipeline script**.
3. Save.

Step 3 Build and Verify

Click **Build Now** and observe the six stages. Verify each stage in the corresponding external system (Section 6

Tomcat 403 Fix — Remote AddrValve

Create the correct pom.xml using printf (no heredoc)

```
printf '%s\n' \
'<?xml version="1.0" encoding="UTF-8"?>' \
'<project xmlns="http://maven.apache.org/POM/4.0.0"' \
'         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"' \
'         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">' \
'  <modelVersion>4.0.0</modelVersion>' \
'  <groupId>com.javatpoint</groupId>' \
```

Jenkins-Task -4

```
'  <artifactId>SimpleCustomerApp</artifactId>' \
'  <packaging>war</packaging>' \
'  <version>${BUILD_NUMBER}-SNAPSHOT</version>' \
'  <name>SimpleCustomerApp</name>' \
'  <url>http://maven.apache.org</url>' \
'  <properties>' \
'     <maven.compiler.source>1.8</maven.compiler.source>' \
'     <maven.compiler.target>1.8</maven.compiler.target>' \
'  </properties>' \
'  <build>' \
'     <plugins>' \
'       <plugin>' \
'         <groupId>org.apache.maven.plugins</groupId>' \
'         <artifactId>maven-war-plugin</artifactId>' \
'         <version>3.3.2</version>' \
'         <configuration>' \
'           <warSourceDirectory>WebContent</warSourceDirectory>' \
'           <failOnMissingWebXml>false</failOnMissingWebXml>' \
'         </configuration>' \
'       </plugin>' \
'     </plugins>' \
'  </build>' \
'</project>' \
> /var/lib/jenkins/workspace/SimpleCustomerApp/pom.xml
```

Verify the first line

```
head -3 /var/lib/jenkins/workspace/SimpleCustomerApp/pom.xml
```

Expected:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
```

Validate XML

```
xmllint --noout /var/lib/jenkins/workspace/SimpleCustomerApp/pom.xml && echo "XML OK"
```

Build the WAR

```
cd /var/lib/jenkins/workspace/SimpleCustomerApp
export JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto.x86_64
/opt/apache-maven-3.10.0/bin/mvn clean package -DBUILD_NUMBER=99
```

Inspect the WAR

Jenkins-Task -4

unzip -l /var/lib/jenkins/workspace/SimpleCustomerApp/target/SimpleCustomerApp-99-SNAPSHOT.war | head -40

Fix ownership back to jenkins so the Jenkins process can still manage the workspace:
chown jenkins:jenkins /var/lib/jenkins/workspace/SimpleCustomerApp/pom.xml
chown -R jenkins:jenkins /var/lib/jenkins/workspace/SimpleCustomerApp/target

Commit and Push
Before pushing, confirm you have push access. Run:
cd /var/lib/jenkins/workspace/SimpleCustomerApp
sudo -u jenkins git config --global --add safe.directory /var/lib/jenkins/workspace/SimpleCustomerApp
sudo -u jenkins git remote -v

Command 1 — Fix ownership
chown jenkins:jenkins /var/lib/jenkins/workspace/SimpleCustomerApp/pom.xml
chown -R jenkins:jenkins /var/lib/jenkins/workspace/SimpleCustomerApp/target
ls -la /var/lib/jenkins/workspace/SimpleCustomerApp/pom.xml

Expected: -rw-r--r--. 1 jenkins jenkins ... pom.xml

Command 2 — Check remote and status as jenkins
sudo -u jenkins git config --global --add safe.directory /var/lib/jenkins/workspace/SimpleCustomerApp
sudo -u jenkins git -C /var/lib/jenkins/workspace/SimpleCustomerApp remote -v
sudo -u jenkins git -C /var/lib/jenkins/workspace/SimpleCustomerApp status
sudo -u jenkins git -C /var/lib/jenkins/workspace/SimpleCustomerApp diff –stat

Before You Push — Get a GitHub Personal Access Token (PAT)

1. Click **Generate new token → Generate new token (classic)**
2. Give it a name like jenkins-simplecustomerapp
3. Set expiration (e.g., 30 days)
4. Check the scope: repo (full control of private repositories)
5. Click **Generate token**
6. **Copy the token immediately**

Replace <YOUR_PAT> below with your actual token (paste it directly, no quotes needed):
sudo -u jenkins git -C /var/lib/jenkins/workspace/SimpleCustomerApp \
push https://<YOUR_PAT>@github.com/Sunith3210/sabear_simplecutomerapp.git feature-1.1

Then Update the Jenkins Pipeline
Stage 1 — Git Clone
stage('Git Clone') {

Jenkins-Task -4

```
    steps {
        git branch: 'feature-1.1',
            url: 'https://github.com/Sunith3210/sabear_simplecutomerapp.git'
    }
}
```

Stage 2 — SonarQube Integration

```
stage('SonarQube Integration') {
    steps {
        script {
            def scannerHome = tool 'Sonar-Scanner'

            withSonarQubeEnv('jenkins_1') {
                sh """
                    ${scannerHome}/bin/sonar-scanner \
                      -Dsonar.projectKey=SimpleCustomerApp \
                      -Dsonar.projectName=SimpleCustomerApp \
                      -Dsonar.sources=. \
                      -Dsonar.projectBaseDir=${WORKSPACE}
                """
            }
        }
    }
}
```

Stage 3 — Maven Compilation

```
stage('Maven Compilation') {
    steps {
        withEnv(["PATH+MAVEN=/opt/apache-maven-3.10.0/bin"]) {
            sh 'mvn -version'
            sh 'mvn clean package'
        }
    }
}
```

Jenkins-Task -4

Stage 4 — Nexus Artifactory

```
stage('Nexus') {
   steps {
      withCredentials([usernamePassword(
         credentialsId: 'nexus_server',
         usernameVariable: 'NEXUS_USERNAME',
         passwordVariable: 'NEXUS_PASSWORD'
      )]) {
         sh '''
            curl -v -u "$NEXUS_USERNAME:$NEXUS_PASSWORD" \
             --upload-file target/*.war \
"http://54.159.123.212:8081/repository/SimpleCustomerApp/com/javatpoint/SimpleCustomerApp/${B
UILD_NUMBER}-SNAPSHOT/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT.war"
         '''
      }
   }
}
```

Stage 5 — Slack Notification

```
stage('Slack Notification') {
   steps {
      slackSend(
         message: "SimpleCustomerApp Build #${BUILD_NUMBER} completed successfully. Nexus upload
completed."
      )
   }
}
```

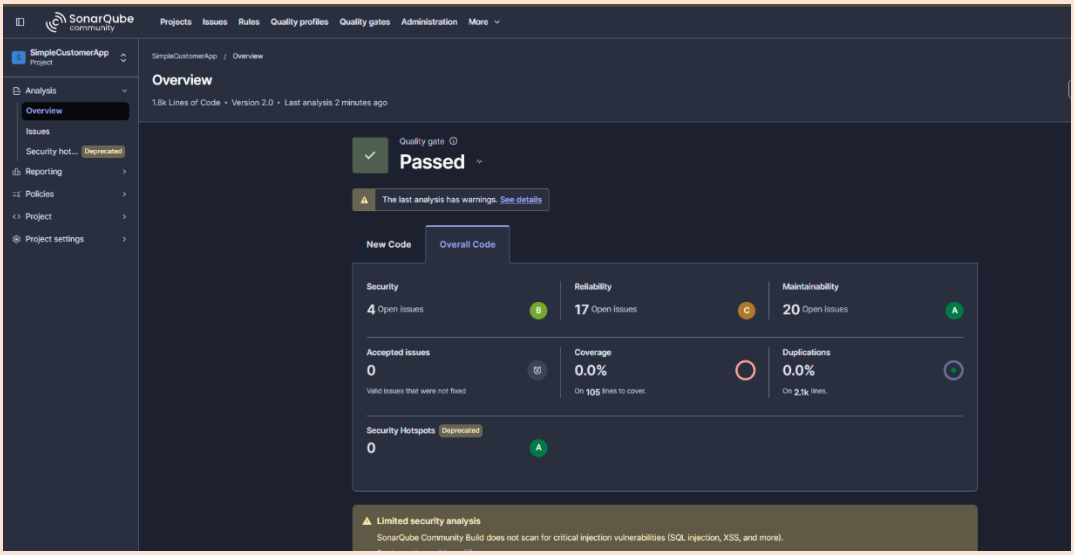
Stage 6 — Deploy On Tomcat

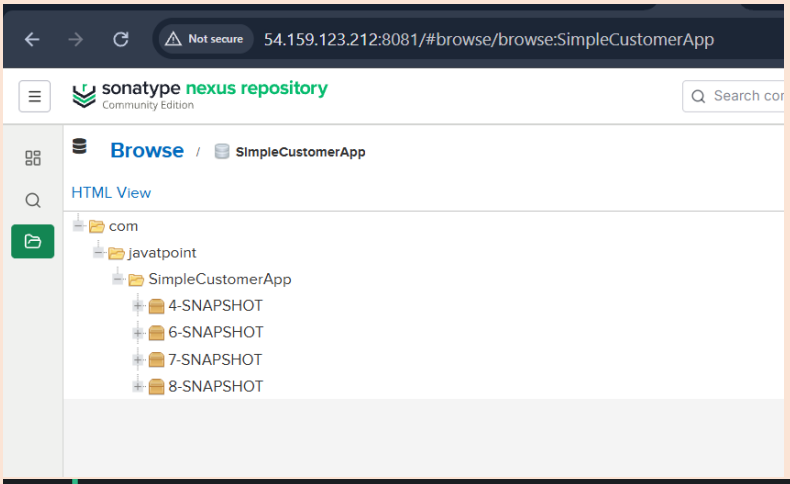
```
stage('Deploy On Tomcat') {
   steps {
      withCredentials([usernamePassword(
         credentialsId: 'tomcat_credentials',
         usernameVariable: 'TOMCAT_USERNAME',
         passwordVariable: 'TOMCAT_PASSWORD'
      )]) {
         sh '''
            curl -v -u "$TOMCAT_USERNAME:$TOMCAT_PASSWORD" \
             --upload-file target/*.war \
             "http://54.82.198.208:8080/manager/text/deploy?path=/SimpleCustomerApp&update=true"
         '''
      }
   }
```

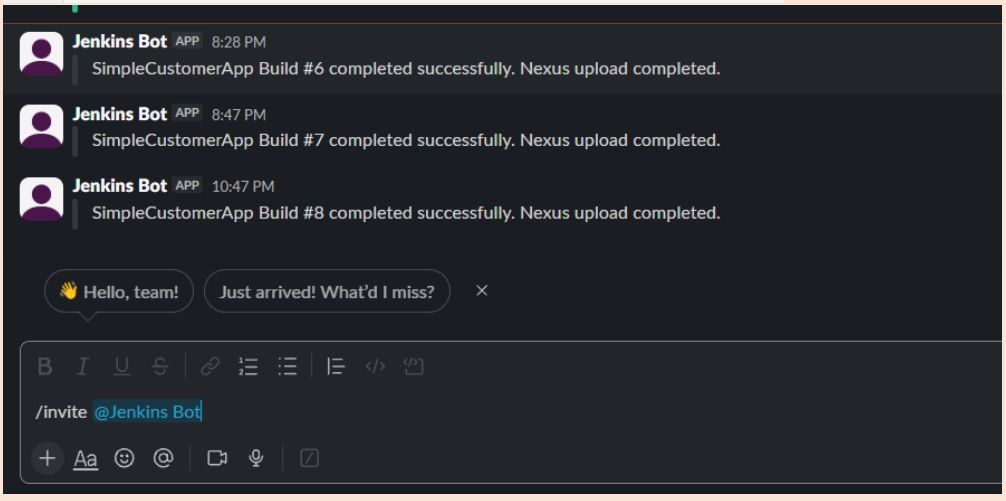
Jenkins-Task -4

}

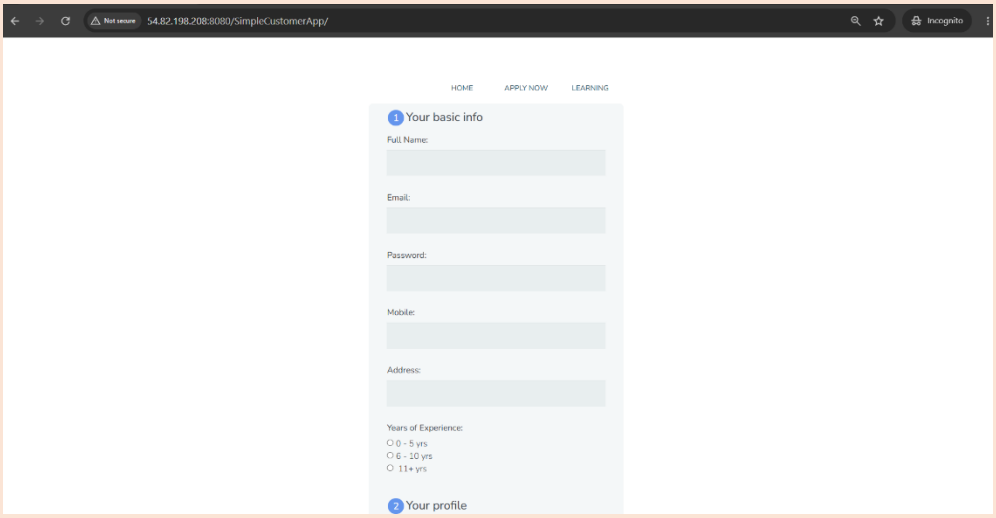
Implementation:







Jenkins-Task -4



3) Setup a Jenkins CICD pipeline using Scripted pipeline using feature-1.1 branch.
Source Code: https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1
stages:

- Git Clone
- SonarQube Integration
- Maven Compilation
- Nexus Artifactory
- Slack Notification
- Deploy On tomcat

Jenkins Job

Create a **new** Jenkins job so Task 2's job stays intact:

1. Jenkins → **New Item**
2. Name: SimpleCustomerApp-Scripted
3. Type: **Pipeline**
4. OK
5. Under **Pipeline** → Definition: **Pipeline script**
6. Paste the script below
7. Save

Objective

To implement a complete **Scripted Jenkins Pipeline** that automates the build, quality analysis, artifact storage, notification, and deployment lifecycle of the SimpleCustomerApp Java web application using the above six stages

Purpose

The purpose of this task is to demonstrate the same six-stage CI/CD workflow as Task 2, but implemented using **Scripted Pipeline** (Groovy DSL) instead of Declarative Pipeline. This highlights the differences in structure, error handling, and syntax between the two Jenkins Pipeline styles, while

Jenkins-Task -4

keeping the underlying CI/CD behaviour identical

Pipeline Script

Stage 1 — Git Clone

```
stage('Git Clone') {
   git branch: 'feature-1.1',
      url: 'https://github.com/Sunith3210/sabear_simplecutomerapp.git'
}
```

Stage 2 — SonarQube Integration

```
stage('SonarQube Integration') {
   scannerHome = tool 'Sonar-Scanner'

   withSonarQubeEnv('jenkins_1') {
      sh """
         ${scannerHome}/bin/sonar-scanner \
           -Dsonar.projectKey=SimpleCustomerApp \
           -Dsonar.projectName=SimpleCustomerApp \
           -Dsonar.sources=. \
           -Dsonar.projectBaseDir=${WORKSPACE}
      """
   }
}
```

Stage 3 — Maven Compilation

```
stage('Maven Compilation') {
   withEnv(["PATH+MAVEN=/opt/apache-maven-3.10.0/bin"]) {
      sh 'mvn -version'
      sh 'mvn clean package'
   }
}
```

Stage 4 — Nexus Artifactory

```
stage('Nexus') {
   withCredentials([usernamePassword(
      credentialsId: 'nexus_server',
      usernameVariable: 'NEXUS_USERNAME',
      passwordVariable: 'NEXUS_PASSWORD'
   )]) {
      sh '''
         curl -v -u "$NEXUS_USERNAME:$NEXUS_PASSWORD" \
          --upload-file target/*.war \
```

Jenkins-Task -4

```
"http://54.159.123.212:8081/repository/SimpleCustomerApp/com/javatpoint/SimpleCustomerApp/${BUILD_NUMBER}-SNAPSHOT/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT.war"
      '''
   }
}
```

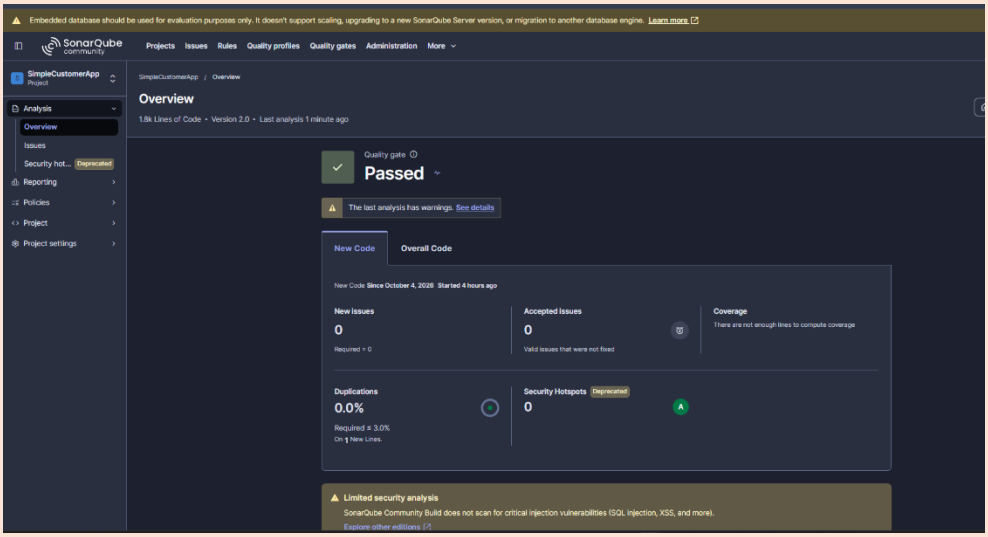
Stage 5 — Slack Notification

```
stage('Slack Notification') {
   slackSend(
      message: "SimpleCustomerApp (Scripted) Build #${BUILD_NUMBER} completed successfully."
   )
}
```

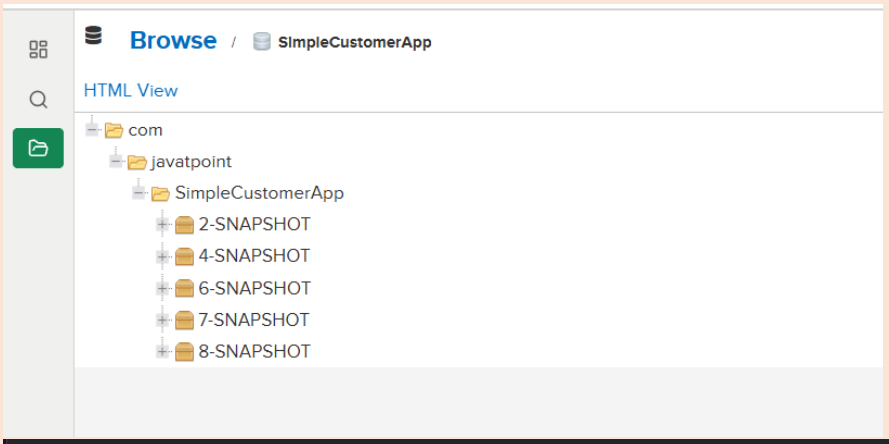
Stage 6 — Deploy On Tomcat

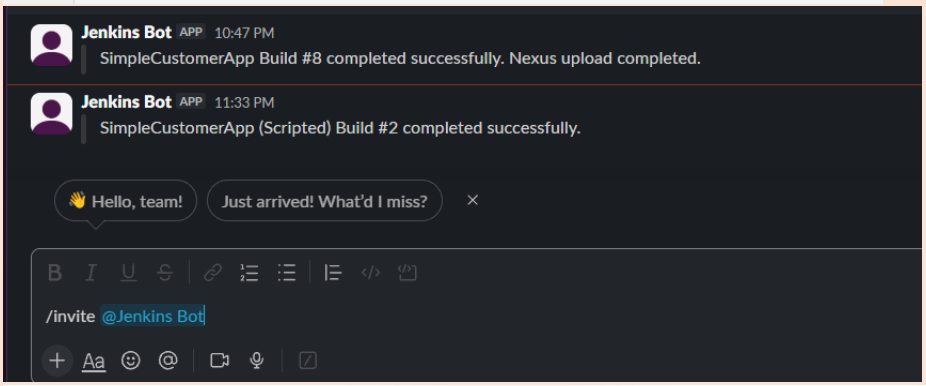
```
stage('Deploy On Tomcat') {
   withCredentials([usernamePassword(
      credentialsId: 'tomcat_credentials',
      usernameVariable: 'TOMCAT_USERNAME',
      passwordVariable: 'TOMCAT_PASSWORD'
   )]) {
      sh '''
         curl -v -u "$TOMCAT_USERNAME:$TOMCAT_PASSWORD" \
           --upload-file target/*.war \
           "http://54.82.198.208:8080/manager/text/deploy?path=/SimpleCustomerApp&update=true"
      '''
   }
}
```

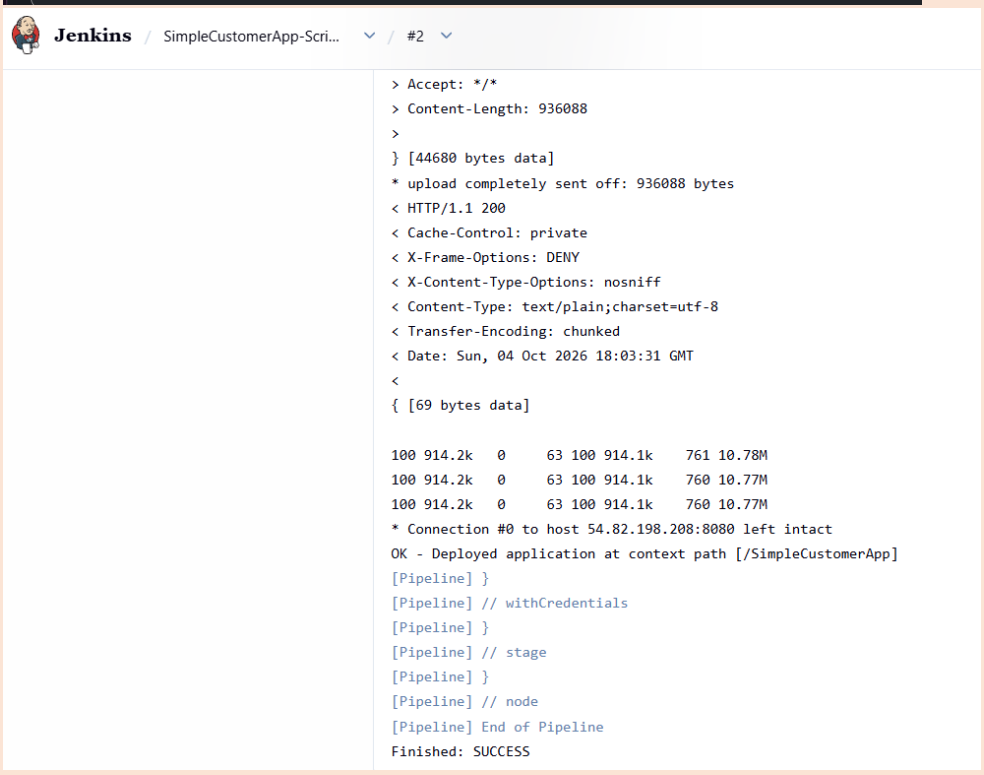
Implementation:



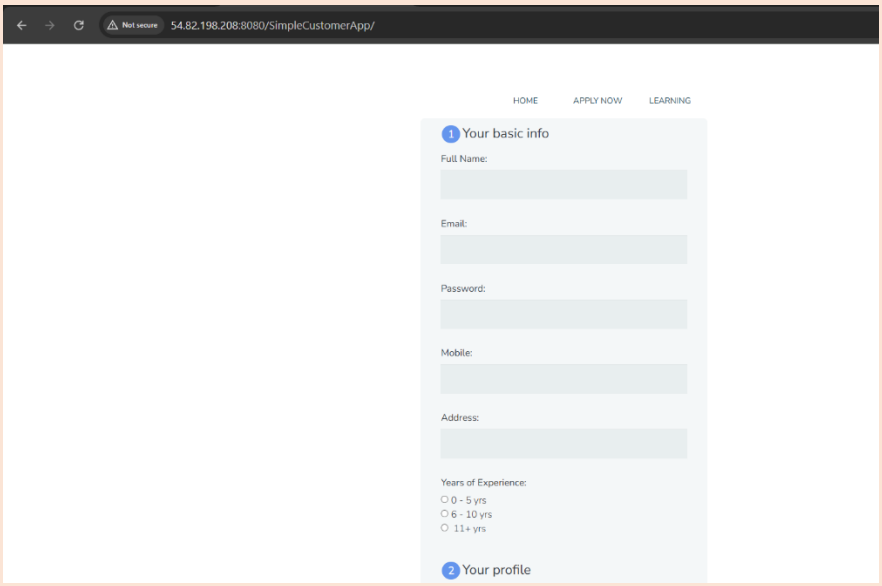
Jenkins-Task -4







Jenkins-Task -4



Conclusion:

final pipeline delivered a working application on Tomcat with a successful Slack notification and Nexus artifact upload.

4) Write sample skeleton of pipelines.

Declarative Skeleton Code

```
pipeline {
  agent any

  environment {
    // Global environment variables would go here
    APP_NAME = 'sample-app'
  }

  options {
    // Pipeline-wide options would go here
    timeout(time: 30, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '10'))
    disableConcurrentBuilds()
  }

  parameters {
    // Job parameters would go here
    string(name: 'BRANCH', defaultValue: 'main', description: 'Git branch to build')
    booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Run tests?')
  }

  triggers {
    // Automatic triggers would go here
```

Jenkins-Task -4

```
    // pollSCM('H/5 * * * *')
}

tools {
    // Tool installations would go here (e.g., maven, jdk)
    // maven 'Maven-3.10.0'
}

stages {

    stage('Checkout') {
        steps {
            echo 'Step 1: Source code checkout would happen here'
            // git branch: "${params.BRANCH}",
            //     url: 'https://github.com/example/repo.git'
        }
    }

    stage('Build') {
        steps {
            echo 'Step 2: Compilation would happen here'
            // sh 'mvn clean package'
        }
    }

    stage('Test') {
        when {
            expression { return params.RUN_TESTS }
        }
        steps {
            echo 'Step 3: Unit tests would run here'
            // sh 'mvn test'
        }
    }

    stage('Code Quality') {
        steps {
            echo 'Step 4: Static analysis would run here'
            // withSonarQubeEnv('jenkins_1') {
            //     sh 'sonar-scanner'
            // }
        }
    }
```

Jenkins-Task -4

```
    stage('Package') {
        steps {
            echo 'Step 5: Artifact packaging would happen here'
            // sh 'mvn package -DskipTests'
        }
    }

    stage('Publish') {
        steps {
            echo 'Step 6: Artifact upload would happen here'
            // nexusArtifactUploader(...)
        }
    }

    stage('Deploy') {
        steps {
            echo 'Step 7: Deployment would happen here'
            // sh 'ansible-playbook deploy.yml'
        }
    }
}

post {
    always {
        echo "Pipeline '${env.JOB_NAME}' build '${env.BUILD_NUMBER}' finished."
    }
    success {
        echo 'Post: Build succeeded.'
        // slackSend(message: "Build ${env.BUILD_NUMBER} succeeded")
    }
    failure {
        echo 'Post: Build failed.'
        // slackSend(message: "Build ${env.BUILD_NUMBER} failed")
    }
    unstable {
        echo 'Post: Build is unstable.'
    }
    aborted {
        echo 'Post: Build was aborted.'
    }
    cleanup {
        echo 'Post: Cleanup actions (e.g., deleteDir()) would run here.'
    }
}
```

Jenkins-Task -4

```
}
```

Scripted Skeleton Code

```
node {

    // -------------------------------------------------------
    // Global variables and defaults
    // -------------------------------------------------------
    def appName       = 'sample-app'
    def branch        = 'main'
    def runTests      = true
    def buildSucceeded = false

    try {

        // -------------------------------------------------------
        // Stage 1: Checkout
        // -------------------------------------------------------
        stage('Checkout') {
            echo 'Step 1: Source code checkout would happen here'
            // git branch: branch,
            //     url: 'https://github.com/example/repo.git'
        }

        // -------------------------------------------------------
        // Stage 2: Build
        // -------------------------------------------------------
        stage('Build') {
            echo 'Step 2: Compilation would happen here'
            // sh 'mvn clean package'
        }

        // -------------------------------------------------------
        // Stage 3: Test
        // -------------------------------------------------------
        stage('Test') {
            if (runTests) {
                echo 'Step 3: Unit tests would run here'
                // sh 'mvn test'
            } else {
                echo 'Step 3: Tests skipped (runTests=false)'
            }
        }
```

Jenkins-Task -4

```
    // ------------------------------------------------------------
    // Stage 4: Code Quality
    // ------------------------------------------------------------
    stage('Code Quality') {
        echo 'Step 4: Static analysis would run here'
        // withSonarQubeEnv('jenkins_1') {
        //     sh 'sonar-scanner'
        // }
    }

    // ------------------------------------------------------------
    // Stage 5: Package
    // ------------------------------------------------------------
    stage('Package') {
        echo 'Step 5: Artifact packaging would happen here'
        // sh 'mvn package -DskipTests'
    }

    // ------------------------------------------------------------
    // Stage 6: Publish
    // ------------------------------------------------------------
    stage('Publish') {
        echo 'Step 6: Artifact upload would happen here'
        // withCredentials([...]) {
        //     sh 'curl --upload-file target/*.war ...'
        // }
    }

    // ------------------------------------------------------------
    // Stage 7: Deploy
    // ------------------------------------------------------------
    stage('Deploy') {
        echo 'Step 7: Deployment would happen here'
        // sh 'ansible-playbook deploy.yml'
    }

    buildSucceeded = true

} catch (Exception e) {

    // ------------------------------------------------------------
    // Failure path
    // ------------------------------------------------------------
    echo "Pipeline failed: ${e.message}"
```

Jenkins-Task -4

```
        // slackSend(message: "Build ${env.BUILD_NUMBER} FAILED: ${e.message}")
        throw e

    } finally {

        // -------------------------------------------------------
        // Always-run block (like Declarative's 'always' / 'cleanup')
        // -------------------------------------------------------
        echo "Pipeline '${env.JOB_NAME}' build '${env.BUILD_NUMBER}' finished. Success:
${buildSucceeded}"
        // deleteDir()
    }
}
```

5) Create a parameterized job in Jenkins.
Source Code: https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git

Objective

To create a Jenkins **parameterized pipeline job** for the spring3-mvc-maven-xml-hello-world-1 Spring MVC application that:

- accepts input parameters at build time,
- conditionally executes stages based on those parameters,
- passes parameter values into the pipeline (e.g., branch selection),
- runs the full CI/CD flow: Git → SonarQube → Maven → Nexus → Slack → Tomcat.4

Purpose

The purpose of this task is to demonstrate Jenkins' **parameterization** feature, which allows a single job to serve multiple build scenarios (branches, environments, optional stages) without editing the pipeline. Parameters expose job-level inputs in the Jenkins UI (Build with Parameters), enabling operational flexibility and reducing pipeline duplication

Step 1 — Create the Parameterized Job

1. Jenkins → **New Item**
2. Name: Spring3-HelloWorld-Parameterized
3. Type: **Pipeline** → OK
4. **Pipeline** → Definition: **Pipeline script**
5. Save

Step 2 — Verify the Parameters Appear

Jenkins-Task -4

After saving, the job page shows a **Build with Parameters** link in the left sidebar. This confirms the parameters were detected.

Step 3 — Run and Verify

1. Click Build with Parameters
2. Accept defaults: BRANCH=master, all booleans true
3. Click Build

Parameters

Name	Type	Default	Purpose
BRANCH	string	master	Git branch to build
RUN_SONAR	boolean	true	Whether to run SonarQube analysis
UPLOAD_NEXUS	boolean	true	Whether to upload the WAR to Nexus
DEPLOY_TOMCAT	boolean	true	Whether to deploy to Tomcat

Pipeline — Stage-by-Stage

Parameter Block

```
parameters {
  string(
    name: 'BRANCH',
    defaultValue: 'master',
    description: 'Git branch to build (e.g., master, pom.xmlversion-testing)'
  )
  booleanParam(
    name: 'RUN_SONAR',
    defaultValue: true,
    description: 'Run SonarQube analysis?'
  )
  booleanParam(
    name: 'UPLOAD_NEXUS',
    defaultValue: true,
    description: 'Upload the WAR to Nexus?'
  )
  booleanParam(
    name: 'DEPLOY_TOMCAT',
    defaultValue: true,
    description: 'Deploy the WAR to Tomcat?'
  )
}
```

Stage 1 — Git Clone

```
stage('Git Clone') {

  steps {

    echo "Cloning branch: ${params.BRANCH}"
```

Jenkins-Task -4

```
    git branch: "${params.BRANCH}",
        url: 'https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git'
  }
}
```

Stage 2 — SonarQube Integration

```
stage('SonarQube Integration') {
  when {
    expression { return params.RUN_SONAR }
  }
  steps {
    script {
      def scannerHome = tool 'Sonar-Scanner'

      withSonarQubeEnv('jenkins_1') {
        sh """
          ${scannerHome}/bin/sonar-scanner \
            -Dsonar.projectKey=spring3-hello-world \
            -Dsonar.projectName=spring3-hello-world \
            -Dsonar.sources=. \
            -Dsonar.projectBaseDir=${WORKSPACE}
        """
      }
    }
  }
}
```

Stage 3 — Maven Compilation

```
stage('Maven Compilation') {
  steps {
    withEnv(["PATH+MAVEN=/opt/apache-maven-3.10.0/bin"]) {
      sh 'mvn -version'
      sh 'mvn clean package -Djdk.version=8'
    }
  }
}
```

Jenkins-Task -4

Stage 4 — Nexus

```
stage('Nexus') {
   when {
      expression { return params.UPLOAD_NEXUS }
   }
   steps {
      withCredentials([usernamePassword(
         credentialsId: 'nexus_server',
         usernameVariable: 'NEXUS_USERNAME',
         passwordVariable: 'NEXUS_PASSWORD'
      )]) {
         sh '''
            WARFILE=$(ls target/*.war | head -1)
            WARNAME=$(basename "$WARFILE")
            echo "Uploading $WARNAME to Nexus"

            curl -v -u "$NEXUS_USERNAME:$NEXUS_PASSWORD" \
              --upload-file "$WARFILE" \
              "http://54.159.123.212:8081/repository/Hiring_App/com/ncodeit/ncodeit-hello-
world/3.0/$WARNAME"
         '''
      }
   }
}
```

Stage 5 — Slack Notification

```
stage('Slack Notification') {
   steps {
      slackSend(
         message: "spring3-hello-world Build #${BUILD_NUMBER} (branch: ${params.BRANCH}) completed
successfully."
      )
   }
}
```

Stage 6 — Deploy On Tomcat

```
stage('Deploy On Tomcat') {

   when {

      expression { return params.DEPLOY_TOMCAT }

   }

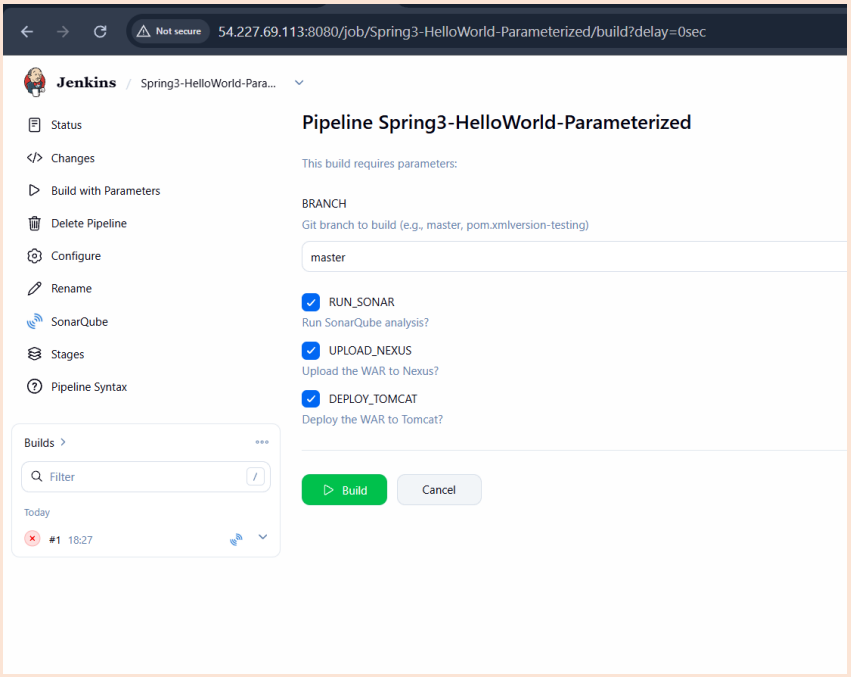
   steps {
```

Jenkins-Task -4

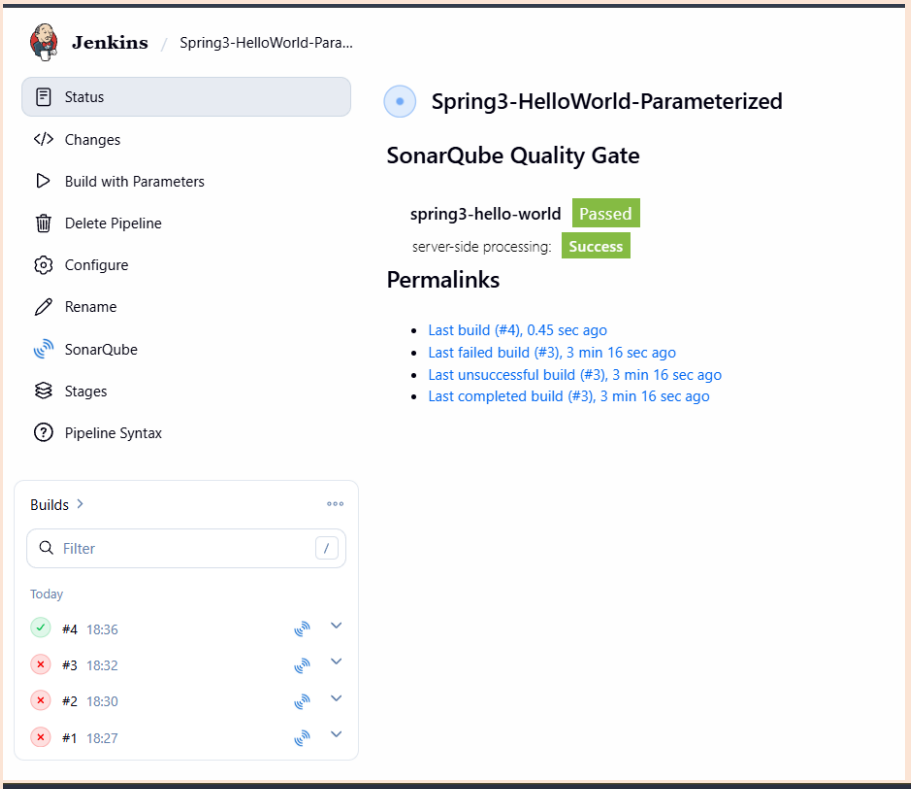
```
        withCredentials([usernamePassword(
            credentialsId: 'tomcat_credentials',
            usernameVariable: 'TOMCAT_USERNAME',
            passwordVariable: 'TOMCAT_PASSWORD'
        )]) {
            sh '''
                WARFILE=$(ls target/*.war | head -1)
                echo "Deploying $WARFILE to Tomcat"

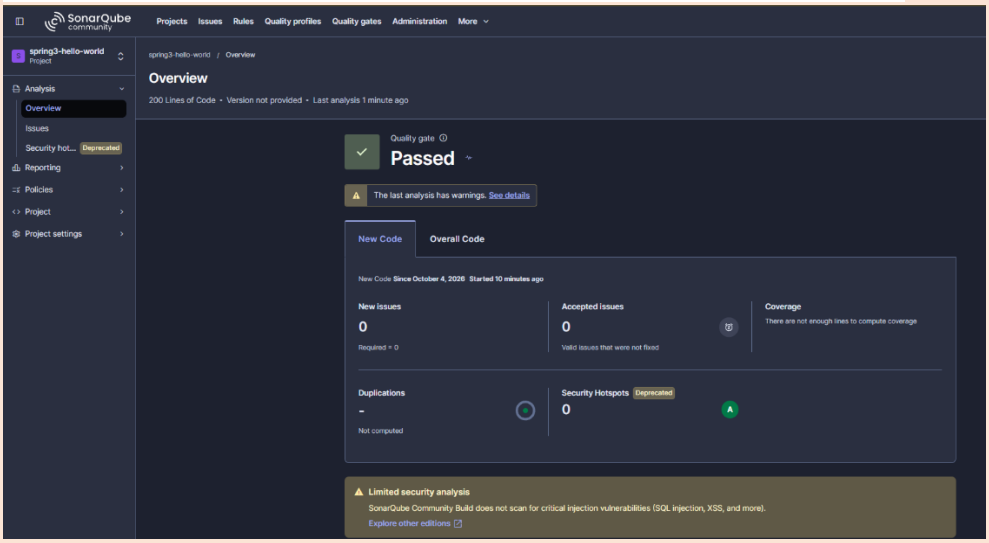
                curl -v -u "$TOMCAT_USERNAME:$TOMCAT_PASSWORD" \
                 --upload-file "$WARFILE" \
                 "http://54.82.198.208:8080/manager/text/deploy?path=/spring3-hello-world&update=true"
            '''
        }
    }
}
```

Implementation:

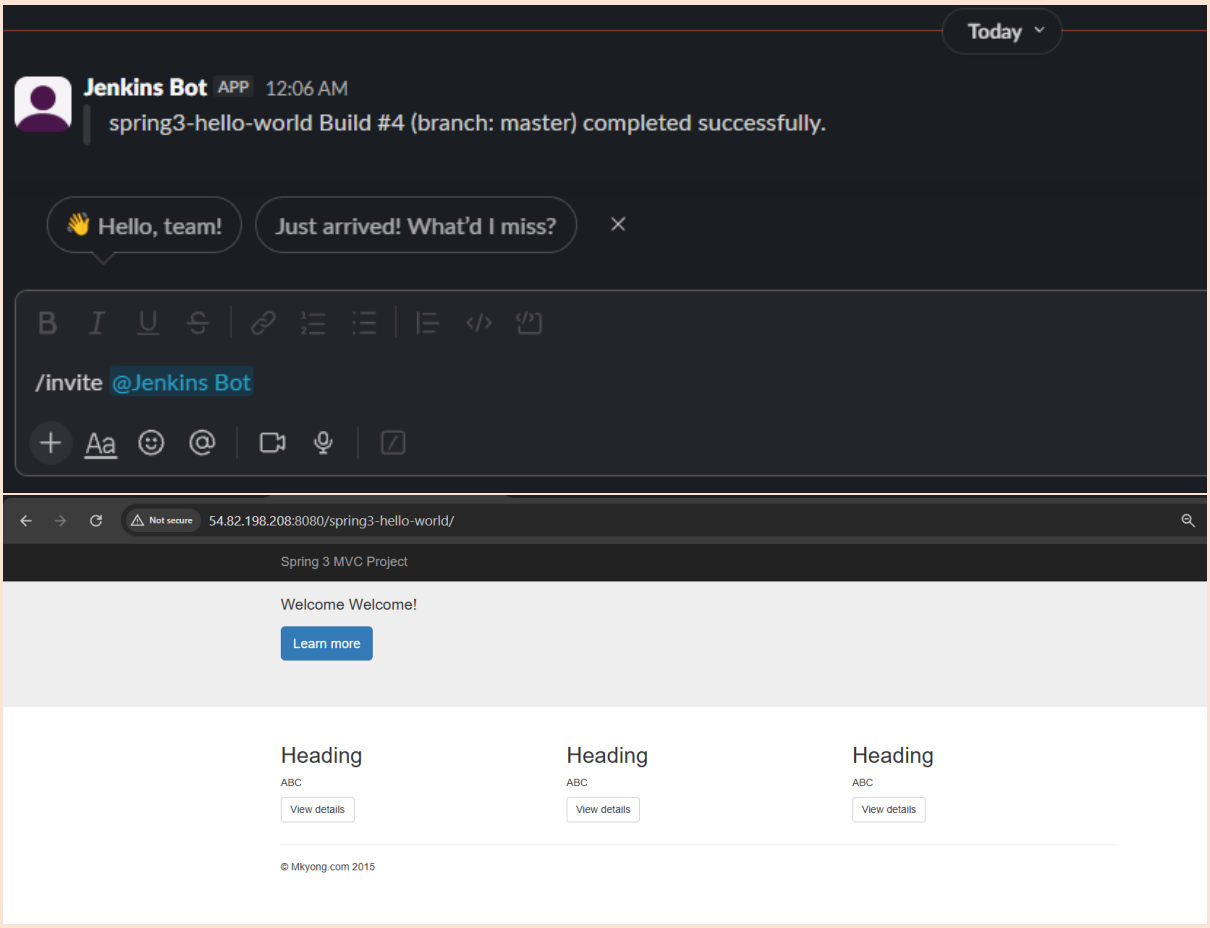


Jenkins-Task -4





Jenkins-Task -4



Conclusion:
Successfully Completed the Task

6) Setup one slave machine for Jenkins.

Jenkins Official Documentation
Recommended

- Pipeline: https://www.jenkins.io/doc/book/pipeline/
- Environment Variables: https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#using-environment-variables
- Parameters: https://www.jenkins.io/doc/book/pipeline/syntax/#parameters
- Handling credentials in Pipeline: https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#handling-credentials

Objective

To set up a **separate Jenkins agent machine** that Jenkins can offload builds to, and to prove that pipeline jobs execute on that agent rather than on the Jenkins controller.

Jenkins-Task -4

Purpose

The purpose of this task is to demonstrate the Jenkins **distributed builds** model, where a controller node orchestrates jobs and one or more agent nodes perform the actual work. This decouples build execution from the controller, allows scaling horizontally, and isolates build workloads. The agent is launched over SSH using a dedicated key and a low-privilege OS user

Step 1 — Generate an SSH Key Pair on the Jenkins Controller

```
sudo -u jenkins mkdir -p /var/lib/jenkins/.ssh
sudo -u jenkins chmod 700 /var/lib/jenkins/.ssh
sudo -u jenkins ssh-keygen -t rsa -b 4096 \
  -f /var/lib/jenkins/.ssh/agent_sonar -N ""
sudo chmod 600 /var/lib/jenkins/.ssh/agent_sonar
sudo chown jenkins:jenkins /var/lib/jenkins/.ssh/agent_sonar*
```

Step 2 — Create a Dedicated Agent User on the Agent EC2

```
sudo useradd -m -s /bin/bash jenkins
sudo passwd -l jenkins
sudo mkdir -p /home/jenkins/.ssh
sudo chmod 700 /home/jenkins/.ssh
```

Add the public key to the new user's authorized_keys:

```
sudo vi /home/jenkins/.ssh/authorized_keys
```

Paste the public key from Master server, then:

```
sudo chown -R jenkins:jenkins /home/jenkins/.ssh
sudo chmod 600 /home/jenkins/.ssh/authorized_keys
```

Optional but recommended — passwordless sudo for the agent user:

```
sudo visudo -f /etc/sudoers.d/Jenkins
```

Add: jenkins ALL=(ALL) NOPASSWD: ALL

Verify SSH From the Controller

```
sudo -u jenkins ssh -i /var/lib/jenkins/.ssh/agent_sonar \
  -o StrictHostKeyChecking=no \
  jenkins@172.31.16.204 "hostname; whoami; java -version"
```

Step 4 — Register the SSH Key as a Jenkins Credential

1. Manage Jenkins → Credentials → System → Global credentials → **Add Credentials**
2. Kind: **SSH Username with private key**
3. ID: sonar-agent-ssh
4. Description: SSH key for SonarQube-EC2 Jenkins agent
5. Username: jenkins

Jenkins-Task -4

6. Private Key: **Enter directly** → paste the contents of /var/lib/jenkins/.ssh/agent_sonar
7. **Create**

Step 5 — Create the Agent Node in Jenkins

1. Manage Jenkins → **Nodes** → **New Node**
2. Name: sonar-agent
3. Type: **Permanent Agent**
4. Fill in

Field	Value
Name	sonar-agent
Description	SonarQube EC2 as Jenkins agent
Number of executors	2
Remote root directory	/home/jenkins
Labels	sonar-agent
Usage	Use this node as much as possible
Launch method	Launch agents via SSH
Host	172.31.16.204
Credentials	sonar-agent-ssh
Host Key Verification Strategy	Non verifying Verification Strategy
Availability	Keep this agent online as much as possible

Save

Test Execution on the Agent

1. Test Execution on the Agent
   Jenkins → New Item → name Agent-Test → Pipeline → OK
2. Pipeline → Definition: Pipeline script
3. Save & Build Now

```
pipeline {
  agent { label 'sonar-agent' }

  stages {
    stage('Where am I') {
      steps {
        sh 'hostname'
        sh 'whoami'
        sh 'java -version'
        sh 'pwd'
      }
    }
  }
}
```

Jenkins-Task -4

Implementation:

